

Opti Crop – Smart Agricultural Production Optimization Engine

Project Description:

Opti Crop is a Machine Learning based crop recommendation system that predicts the most suitable crop using soil nutrients and environmental conditions. The application takes Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall as inputs and recommends the best crop using the K-Nearest Neighbours (KNN) algorithm. Developed using Flask, HTML, CSS, and Scikit-Learn, the system provides farmers with fast and accurate crop recommendations.

Scenarios:

Scenario 1

A farmer enters the soil nutrient values (Nitrogen, Phosphorous, Potassium) along with temperature, humidity, pH, and rainfall into the Opti Crop web application. The system processes the inputs using the KNN Machine Learning model and recommends the most suitable crop for cultivation.

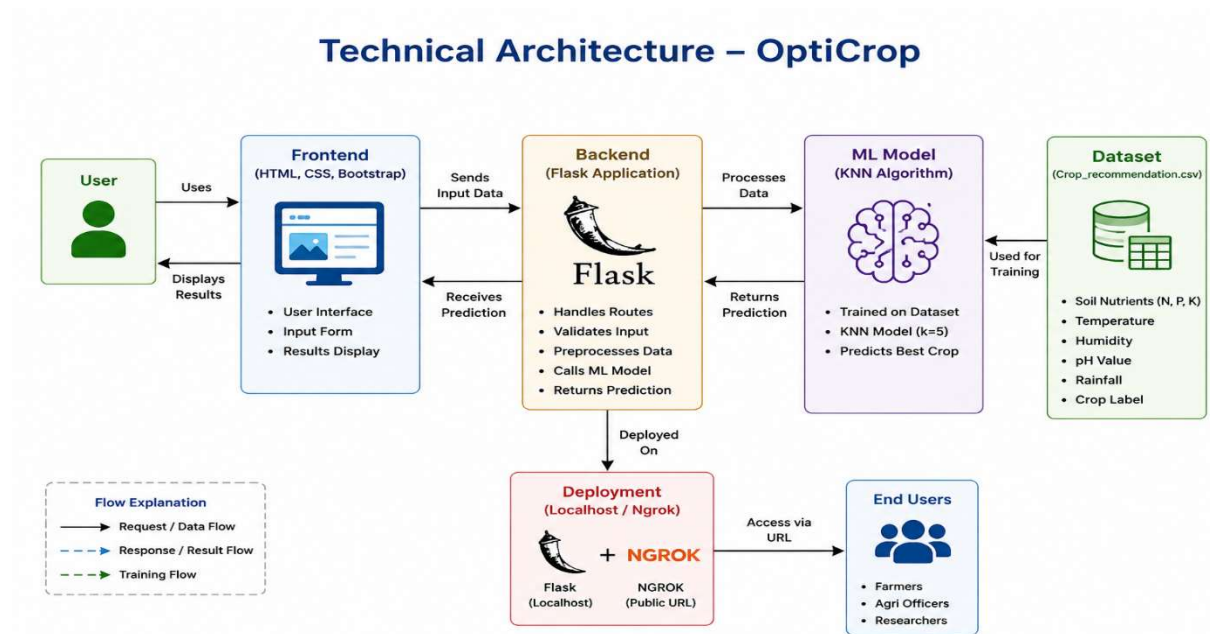
Scenario 2

An agricultural officer analyses different soil and weather conditions from multiple farms. By using Opti Crop, the officer quickly obtains crop recommendations, helping farmers improve productivity and make better cultivation decisions.

Scenario 3

A beginner farmer who is unsure which crop to cultivate uses the Opti Crop application. After entering the required agricultural parameters, the system instantly predicts the best crop, reducing the chances of poor crop selection and improving farming efficiency.

Technical Architecture:



Description: The Opti Crop system allows users to enter soil and environmental parameters through a web interface. The Flask backend receives the input and sends it to the trained KNN machine learning model. The model predicts the most suitable crop using the agricultural dataset and returns the result, which is displayed to the user through the web application.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

"Not Applicable (No database used in the project)."

Pre-requisites:

- **Python Programming Knowledge:** Python Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Basics (KNN):** Scikit-Learn Documentation
- **NumPy & Pandas Familiarity:** NumPy and Pandas Documentation
- **HTML & CSS Skills:** W3Schools HTML/CSS Tutorials
- **Version Control with Git & GitHub:** Git Documentation
- **Development Environment Setup:** Visual Studio Code & Python Installation Guide
- **Crop Recommendation Dataset:** Kaggle Crop Recommendation Dataset
- **Web Browser for Testing:** Google Chrome / Microsoft Edge

Project Workflow:

Milestone 1: Dataset Collection and Analysis

Activity 1.1: Collect the Crop Recommendation dataset containing soil nutrients and environmental parameters.

Activity 1.2: Analyse the dataset and understand the features required for crop prediction.

Activity 1.3: Perform data preprocessing by checking missing values and preparing the dataset.

Activity 1.4: Set up the development environment by installing Python, Flask, Scikit-Learn, and other required libraries.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Split the dataset into training and testing datasets.
- **Activity 2.2:** Train the K-Nearest Neighbours (KNN) model for crop prediction.
- **Activity 2.3:** Evaluate the model performance and save the trained model using Pickle.

Milestone 3: Flask Application Development

- **Activity 3.1:** Develop the main application logic in app.py.
- **Activity 3.2:** Create Flask routes for Home, About, Crop Recommendation, and Prediction pages.
- **Activity 3.3:** Integrate the trained KNN model with the Flask application.

Milestone 4: Frontend Development

- **Activity 4.1:** Design responsive web pages using HTML and CSS.
- **Activity 4.2:** Develop user-friendly forms for entering agricultural parameters.
- **Activity 4.3:** Display crop prediction results through the web interface.

Milestone 5: Testing and Validation

- **Activity 5.1:** Test the application locally using the Flask development server.
- **Activity 5.2:** Perform performance testing using Apache JMeter and verify application functionality

Milestone 6: Conclusion

- **Activity 6.1:** Validate the overall system performance and functionality.
- **Activity 6.2:** Complete project documentation and prepare the project for GitHub submission.

Milestone 1: Dataset Collection and Analysis

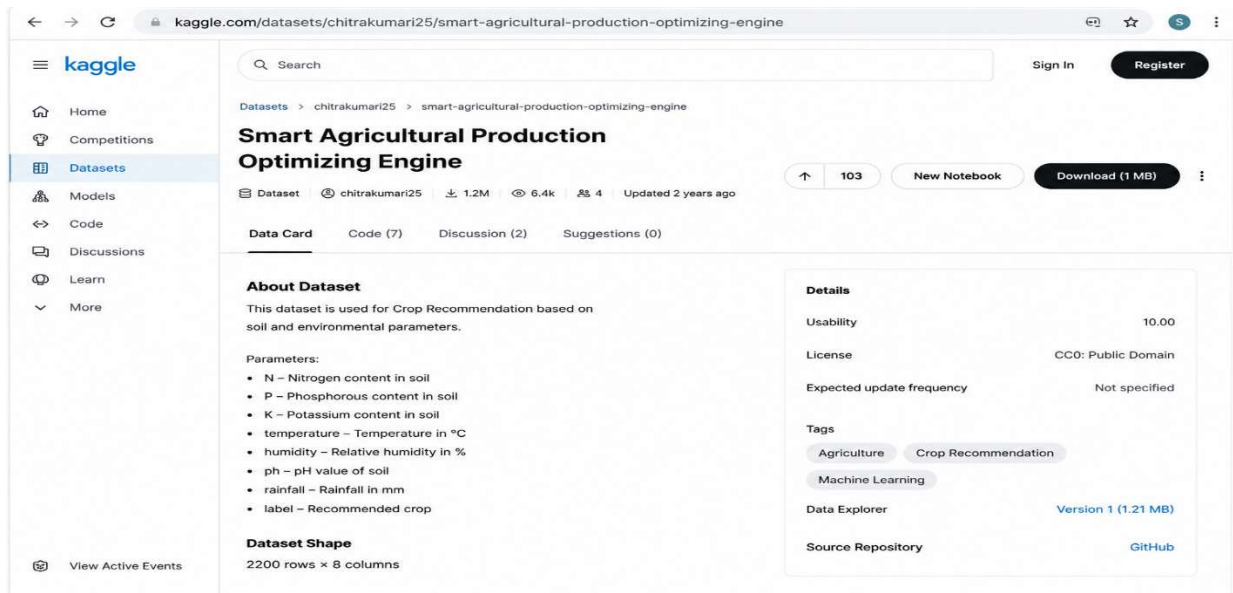
In this milestone, we collect the Crop Recommendation dataset, analyse its features, preprocess the data, and prepare the development environment for building the Machine Learning model.

Activity 1.1: Collect the Crop Recommendation Dataset

Before developing the crop recommendation model, a suitable agricultural dataset must be collected. The Crop Recommendation dataset contains important soil nutrients and environmental parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH, Rainfall, and Crop Label. This dataset is used for training and evaluating the KNN Machine Learning model.

Steps

- **Step 1:** Open the Kaggle website.
- **Step 2:** Search for **Crop Recommendation Dataset**.
- **Step 3:** Download the dataset in CSV format.
- **Step 4:** Save the dataset inside the **dataset** folder.



The screenshot shows the Kaggle dataset page for "Smart Agricultural Production Optimizing Engine" by user chitrakumari25. The dataset is 1.2M in size, has 6.4k views, and 4 downloads. It was updated 2 years ago. The page includes a sidebar with navigation options like Home, Competitions, Datasets, Models, Code, Discussions, Learn, and More. The main content area shows the dataset title, a search bar, and a "Download (1 MB)" button. Below the title, there's a "Data Card" section with "About Dataset" information, "Parameters" (N, P, K, temperature, humidity, pH, rainfall, label), and "Dataset Shape" (2200 rows x 8 columns). A "Details" section on the right shows Usability (10.00), License (CC0: Public Domain), Expected update frequency (Not specified), Tags (Agriculture, Crop Recommendation, Machine Learning), Data Explorer (Version 1 (1.21 MB)), and Source Repository (GitHub).

Activity 1.2: Explore and Analyze the Dataset

After downloading the dataset, it is loaded into Jupyter Notebook using Pandas. The dataset structure, feature names, data types, and sample records are examined to understand the available agricultural parameters before preprocessing.

Steps

- **Step 1:** Import the Pandas library.
- **Step 2:** Load the dataset using `pd.read_csv()`.
- **Step 3:** Display the first few rows using `head ()`.
- **Step 4:** Check dataset information using `info ()`

```

jupyter Crop_Recommendation_Analysis Last Checkpoint: a minute ago (autosaved)
File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

In [1]: import pandas as pd
df = pd.read_csv('Crop_recommendation.csv')

In [2]: df.head()
Out[2]:
   N      P      K  temperature  humidity      ph  rainfall  label
0   90     42     43    29.879744    82.002744    6.50295    202.935536  rice
1   85     58     41    21.770462    80.319644    7.038096    226.655537  rice
2   60     55     44    23.004459    82.320763    7.840207    263.964248  rice
3   74     35     40    26.491096    80.158363    6.988401    242.864034  rice
4   78     42     42    20.130175    81.604873    7.628473    262.717340  rice

In [3]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   N            2200 non-null    int64
1   P            2200 non-null    int64
2   K            2200 non-null    int64
3   temperature  2200 non-null    float64
4   humidity     2200 non-null    float64
5   ph           2200 non-null    float64
6   rainfall     2200 non-null    float64
7   label        2200 non-null    object
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB

```

Activity 1.3: Data Preprocessing

The dataset is pre-processed by checking missing values, verifying data consistency, and preparing the features for model training. Clean and well-structured data improves the performance of the Machine Learning model.

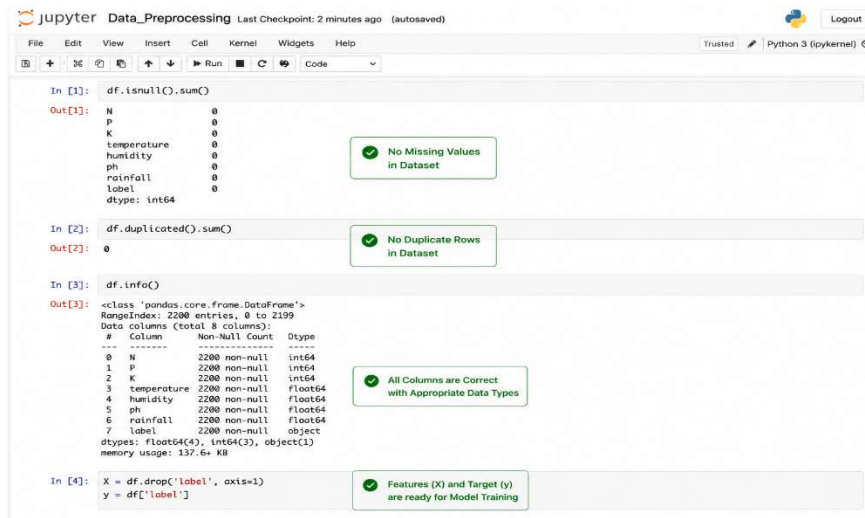
Steps

Step 1: Check missing values using `is null().sum()`.

Step 2: Verify data types.

Step 3: Identify duplicate records.

Step 4: Prepare input and output variables.



The image shows a Jupyter Notebook interface with the following content:

```

In [1]: df.isnull().sum()
Out[1]:
N      0
P      0
K      0
temperature      0
humidity      0
ph      0
rainfall      0
label      0
dtype: int64
No Missing Values in Dataset

In [2]: df.duplicated().sum()
Out[2]: 0
No Duplicate Rows in Dataset

In [3]: df.info()
Out[3]:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0    N      2200 non-null    int64
1    P      2200 non-null    int64
2    K      2200 non-null    int64
3    temperature  2200 non-null    float64
4    humidity    2200 non-null    float64
5    ph          2200 non-null    float64
6    rainfall    2200 non-null    float64
7    label       2200 non-null    object
dtypes: float64(4), int64(2), object(1)
memory usage: 137.6+ KB
All Columns are Correct with Appropriate Data Types

In [4]: X = df.drop('label', axis=1)
        y = df['label']
Features (X) and Target (y) are ready for Model Training
  
```

Activity 1.4: Development Environment Setup

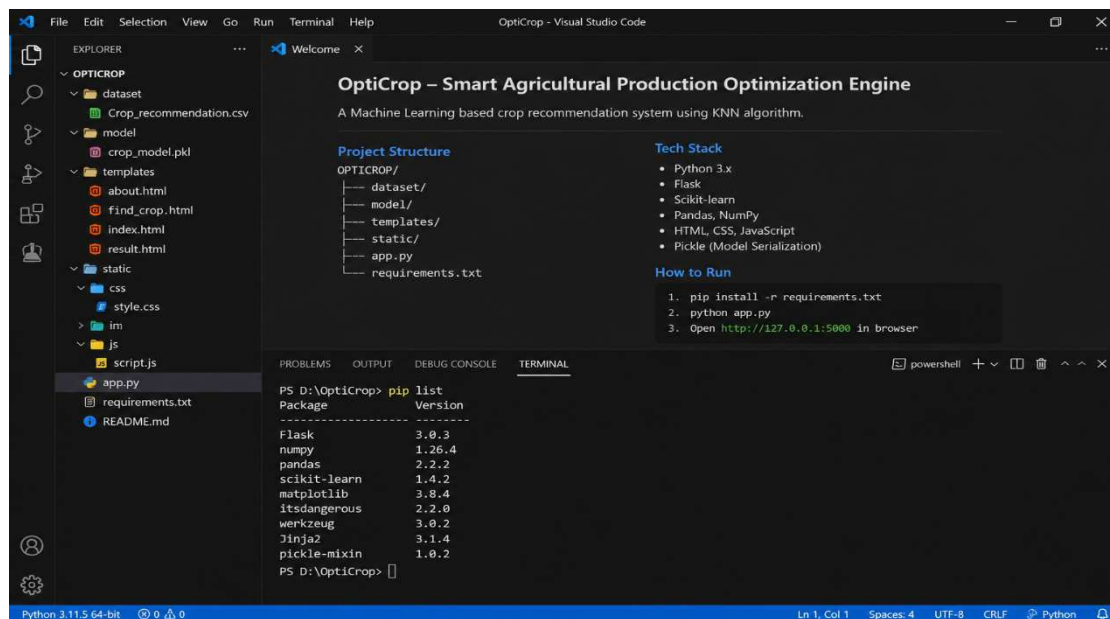
The required software tools and Python libraries are installed to build the Machine Learning application. Flask is used for the web application, while Scikit-Learn is used for model development.

Steps

- **Step 1:** Install Python 3.13 or above.
- **Step 2:** Install Visual Studio Code.
- **Step 3:** Install required libraries using:

```
pip install flask NumPy pandas scikit-learn matplotlib
```

- **Step 4:** Create the project folders and open the project in VS Code.



Activity 2.1: Split the Dataset into Training and Testing Sets

The dataset is divided into training and testing datasets to build and evaluate the Machine Learning model. The training dataset is used to train the KNN model, while the testing dataset is used to measure its prediction accuracy.

Steps

Step 1: Import train_test_split from Scikit-Learn.

Step 2: Divide the dataset into training and testing sets.

Step 3: Verify the size of the generated datasets.

Activity 2.2: Train the KNN Model

The K-Nearest Neighbours (KNN) algorithm is selected and trained using the training dataset. The model learns relationships between soil nutrients, environmental conditions, and crop labels to provide accurate crop recommendations.

Steps:

- **Step 1:** Import KNeighborsClassifier.
- **Step 2:** Create the KNN model.
- **Step 3:** Train the model using the training dataset.

Activity 2.3: Evaluate and Save the Model

The trained KNN model is evaluated using the testing dataset to verify its prediction performance. After evaluation, the trained model is saved as `crop_model.pkl` using the Pickle library for deployment in the Flask application.

Steps:

- **Step 1:** Predict crop labels using the testing data.
- **Step 2:** Evaluate model accuracy.
- **Step 3:** Save the trained model as `crop_model.pkl`.

MILESTONE 2: MODEL DEVELOPMENT AND EVALUATION

In this milestone, the dataset is split into training and testing sets, the KNN model is trained, and the model is evaluated and saved for future predictions.

Activity 2.1: Split the Dataset into Training and Testing Sets

Matter: The dataset is divided into training and testing sets. The training set is used to train the KNN model and the testing set is used to evaluate its performance.

Steps:

- **Step 1:** Import `train_test_split` from `sklearn`.
- **Step 2:** Split the data into 80% training and 20% testing.
- **Step 3:** Display the shapes of training and testing sets.

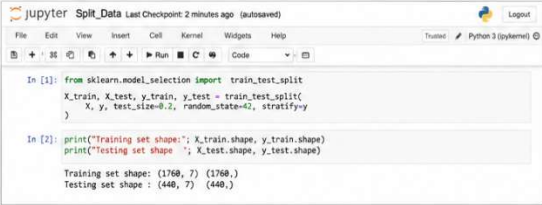


Image: Train-Test Split of the Dataset.

Activity 2.2: Train the KNN Model

Matter: The K-Nearest Neighbors (KNN) algorithm is trained using the training data. The model learns patterns between soil parameters and crop labels.

Steps:

- **Step 1:** Import `KNeighborsClassifier`.
- **Step 2:** Create a KNN model with `k = 5`.
- **Step 3:** Train the model using training data.

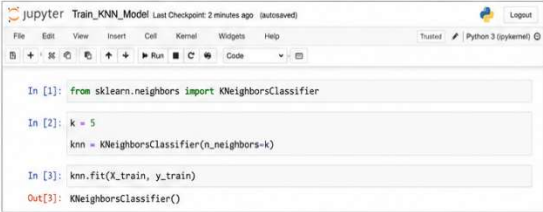


Image: Training the KNN Model.

Activity 2.3: Evaluate the Model and Save the Model

Matter: The trained model is evaluated using various metrics such as accuracy, precision, recall, and F1-score. The model is then saved using pickle for future use.

Steps:

- **Step 1:** Predict the output on test data.
- **Step 2:** Calculate evaluation metrics.
- **Step 3:** Save the trained model using pickle.

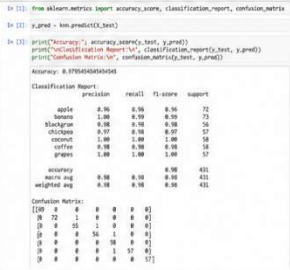



Image: Model Evaluation Metrics and Model Saved.

Summary of Milestone 2

- ✓ Dataset is split into training and testing sets.
- ✓ KNN model is trained using training data.
- ✓ Model is evaluated using accuracy, precision, recall, F1-score, and confusion matrix.
- ✓ Trained model is saved as `crop_model.pkl` for deployment.

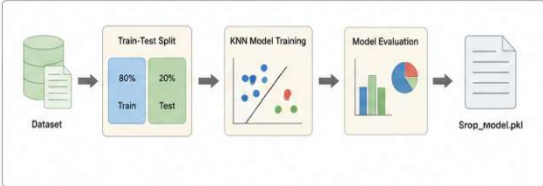


Image: Milestone 2 Workflow.

Milestone 3: Flask Application Development

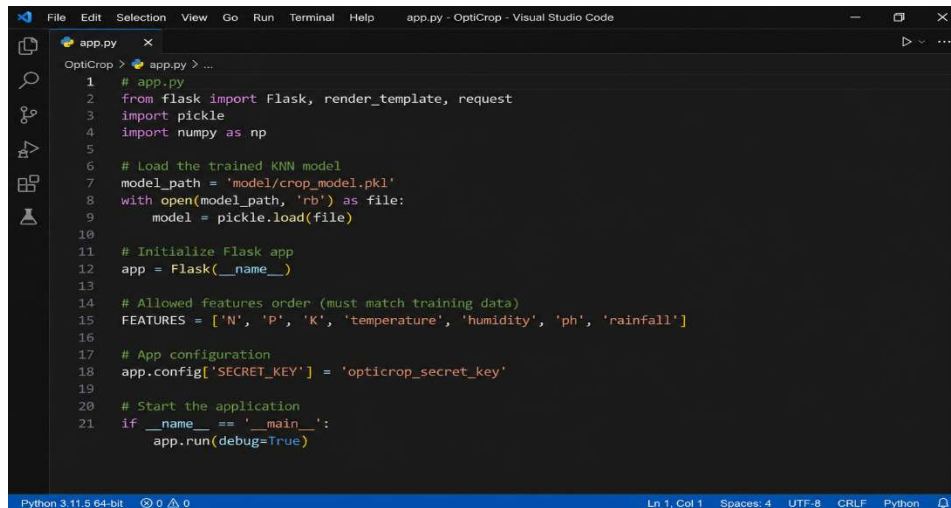
In this milestone, the Flask web application is developed to integrate the trained KNN model with the user interface. The application receives agricultural parameters from users, processes the inputs, predicts the most suitable crop, and displays the result through interactive web pages.

Activity 3.1: Develop the Main Application (app.py)

The main application logic is implemented in **app.py**. Flask routes are created to manage different web pages, load the trained KNN model, receive user inputs, and generate predictions.

Steps:

- **Step 1:** Import Flask and required Python libraries.
- **Step 2:** Load the trained KNN model (crop_model.pkl).
- **Step 3:** Create the Flask application.
- **Step 4:** Define application routes.



```

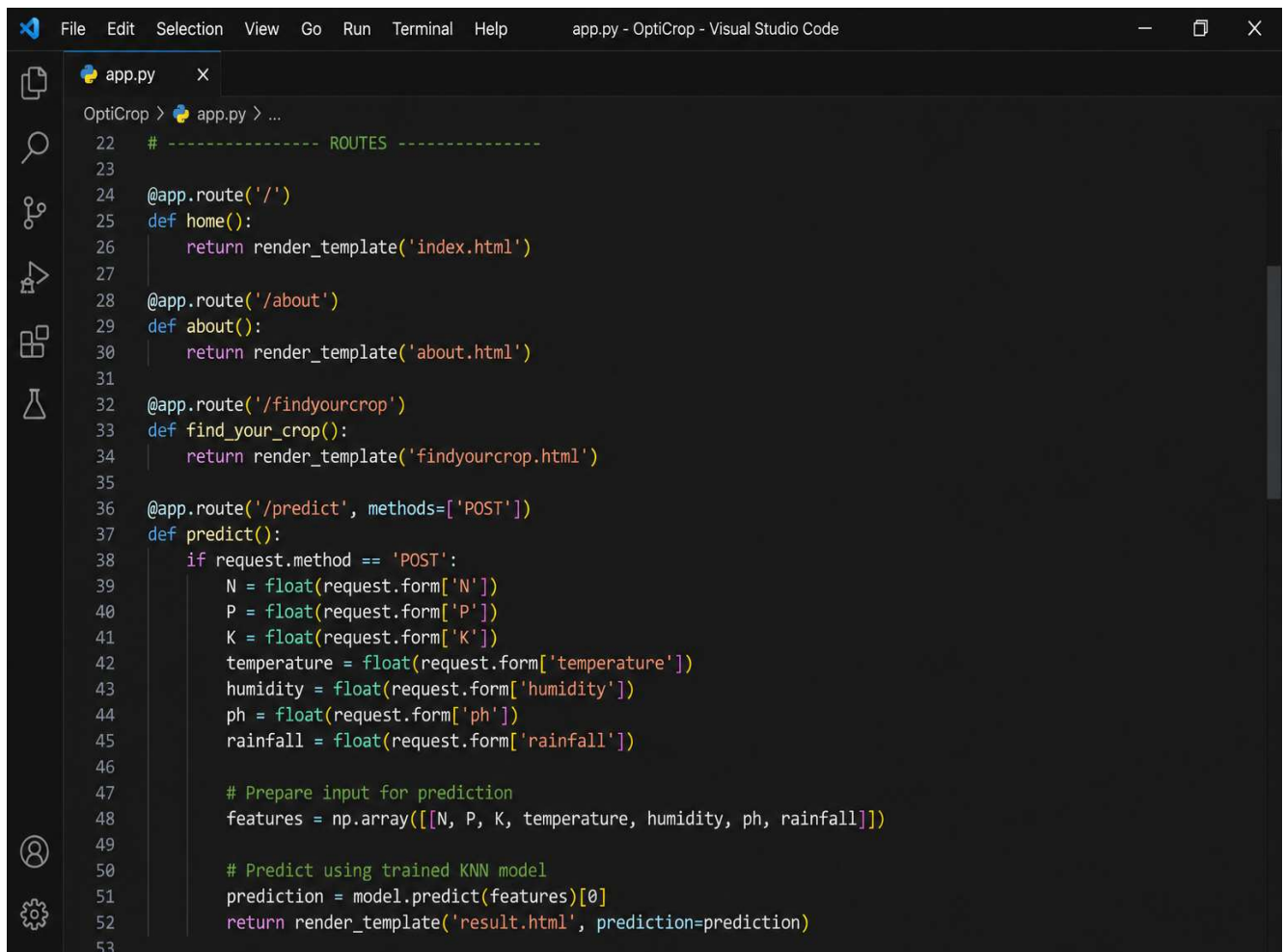
1  # app.py
2  from flask import Flask, render_template, request
3  import pickle
4  import numpy as np
5
6  # Load the trained KNN model
7  model_path = 'model/crop_model.pkl'
8  with open(model_path, 'rb') as file:
9      model = pickle.load(file)
10
11 # Initialize Flask app
12 app = Flask(__name__)
13
14 # Allowed features order (must match training data)
15 FEATURES = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
16
17 # App configuration
18 app.config['SECRET_KEY'] = 'opticrop_secret_key'
19
20 # Start the application
21 if __name__ == '__main__':
22     app.run(debug=True)
  
```

Activity 3.2: Create Flask Routes

Multiple Flask routes are created to provide navigation between the Home, About, Find Your Crop, and Result pages. These routes improve the user experience and organize the application structure.

Steps:

- **Step 1:** Create the Home route (/).
- **Step 2:** Create the About route (/about).
- **Step 3:** Create the Find Your Crop route (/findyourcrop).
- **Step 4:** Create the Prediction route (/predict).



```
File Edit Selection View Go Run Terminal Help app.py - OptiCrop - Visual Studio Code

app.py x
OptiCrop > app.py > ...

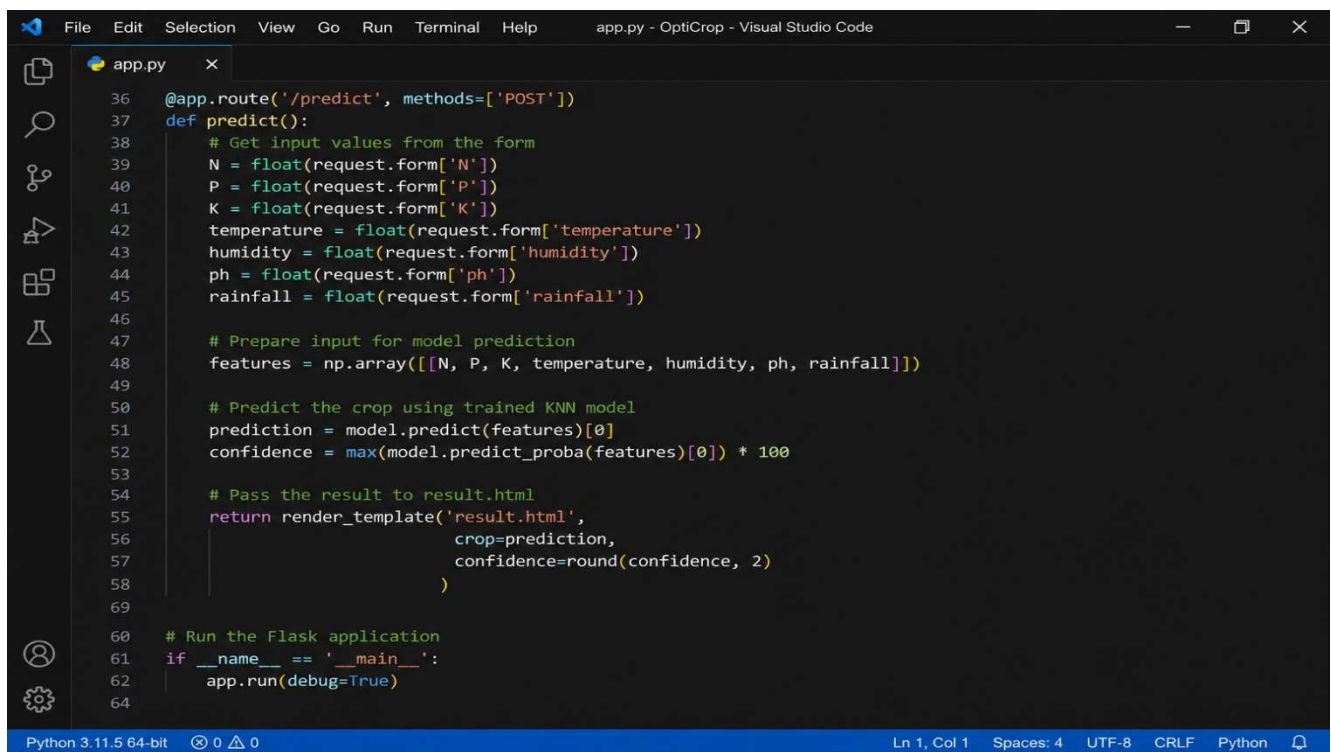
22 # ----- ROUTES -----
23
24 @app.route('/')
25 def home():
26     return render_template('index.html')
27
28 @app.route('/about')
29 def about():
30     return render_template('about.html')
31
32 @app.route('/findyourcrop')
33 def find_your_crop():
34     return render_template('findyourcrop.html')
35
36 @app.route('/predict', methods=['POST'])
37 def predict():
38     if request.method == 'POST':
39         N = float(request.form['N'])
40         P = float(request.form['P'])
41         K = float(request.form['K'])
42         temperature = float(request.form['temperature'])
43         humidity = float(request.form['humidity'])
44         ph = float(request.form['ph'])
45         rainfall = float(request.form['rainfall'])
46
47         # Prepare input for prediction
48         features = np.array([[N, P, K, temperature, humidity, ph, rainfall]])
49
50         # Predict using trained KNN model
51         prediction = model.predict(features)[0]
52         return render_template('result.html', prediction=prediction)
53
```

Activity 3.3: Integrate the KNN Model

The prediction route accepts agricultural parameters entered by the user, converts them into the required format, sends them to the trained KNN model, and displays the recommended crop on the result page.

Steps:

- **Step 1:** Read user inputs from the HTML form.
- **Step 2:** Convert the inputs into a NumPy array.
- **Step 3:** Predict the crop using the KNN model.
- **Step 4:** Display the prediction on the result page.



```

36 @app.route('/predict', methods=['POST'])
37 def predict():
38     # Get input values from the form
39     N = float(request.form['N'])
40     P = float(request.form['P'])
41     K = float(request.form['K'])
42     temperature = float(request.form['temperature'])
43     humidity = float(request.form['humidity'])
44     ph = float(request.form['ph'])
45     rainfall = float(request.form['rainfall'])
46
47     # Prepare input for model prediction
48     features = np.array([[N, P, K, temperature, humidity, ph, rainfall]])
49
50     # Predict the crop using trained KNN model
51     prediction = model.predict(features)[0]
52     confidence = max(model.predict_proba(features)[0]) * 100
53
54     # Pass the result to result.html
55     return render_template('result.html',
56                           crop=prediction,
57                           confidence=round(confidence, 2)
58     )
59
60 # Run the Flask application
61 if __name__ == '__main__':
62     app.run(debug=True)
63
64

```

Python 3.11.5 64-bit 0 0 0 Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python

Milestone 4: Frontend Development

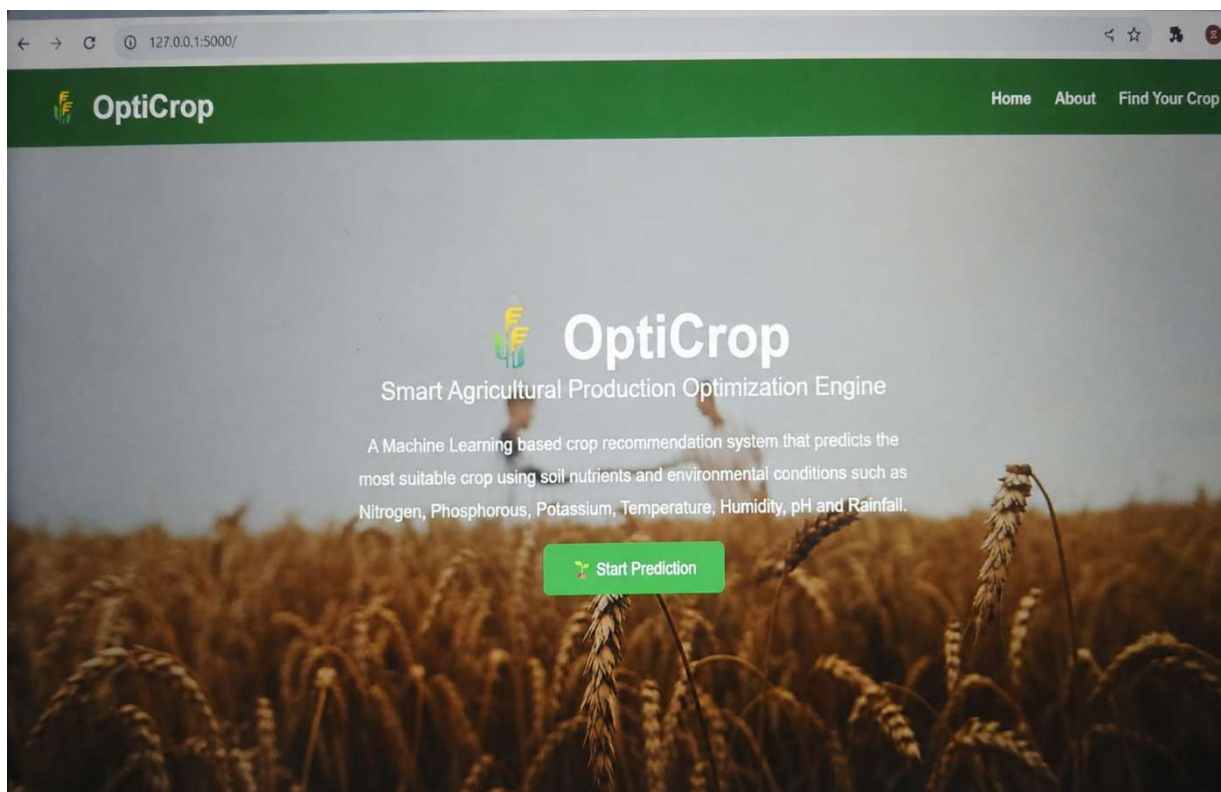
In this milestone, the user interface of the Opti Crop application is developed using HTML and CSS. The web pages are designed to provide a simple, responsive, and user-friendly experience for entering agricultural parameters and viewing crop prediction results.

Activity 4.1: Design the Home Page

The Home page is developed to introduce the Opti Crop project. It provides project information, navigation links, and a button to start the crop recommendation process.

Steps:

- **Step 1:** Create home.html.
- **Step 2:** Design the navigation bar.
- **Step 3:** Add the project title and description.
- **Step 4:** Add the Start Prediction button.

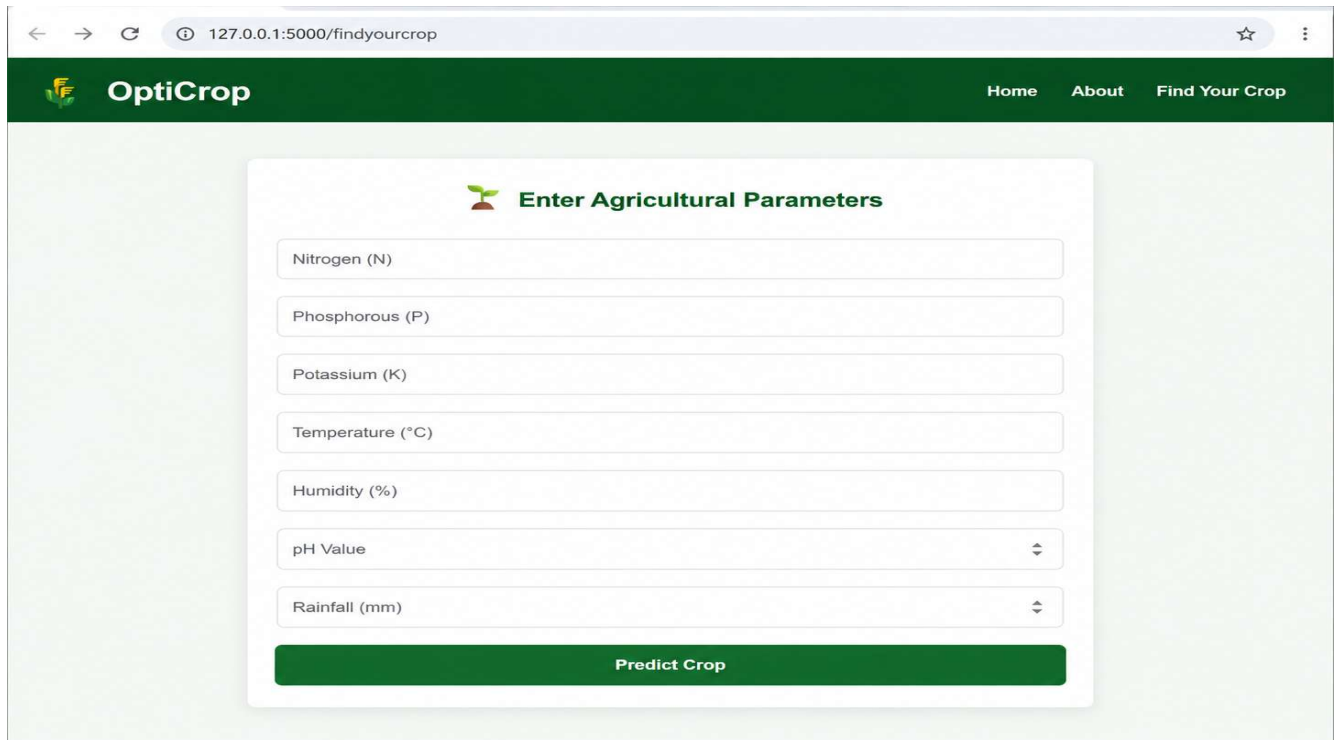


Activity 4.2: Develop the Crop Prediction Form

A responsive form is designed to collect agricultural parameters such as Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall from the user.

Steps:

- **Step 1:** Create findyourcrop.html.
- **Step 2:** Add input fields for all parameters.
- **Step 3:** Design the Predict button.
- **Step 4:** Connect the form to the Flask /predict route.



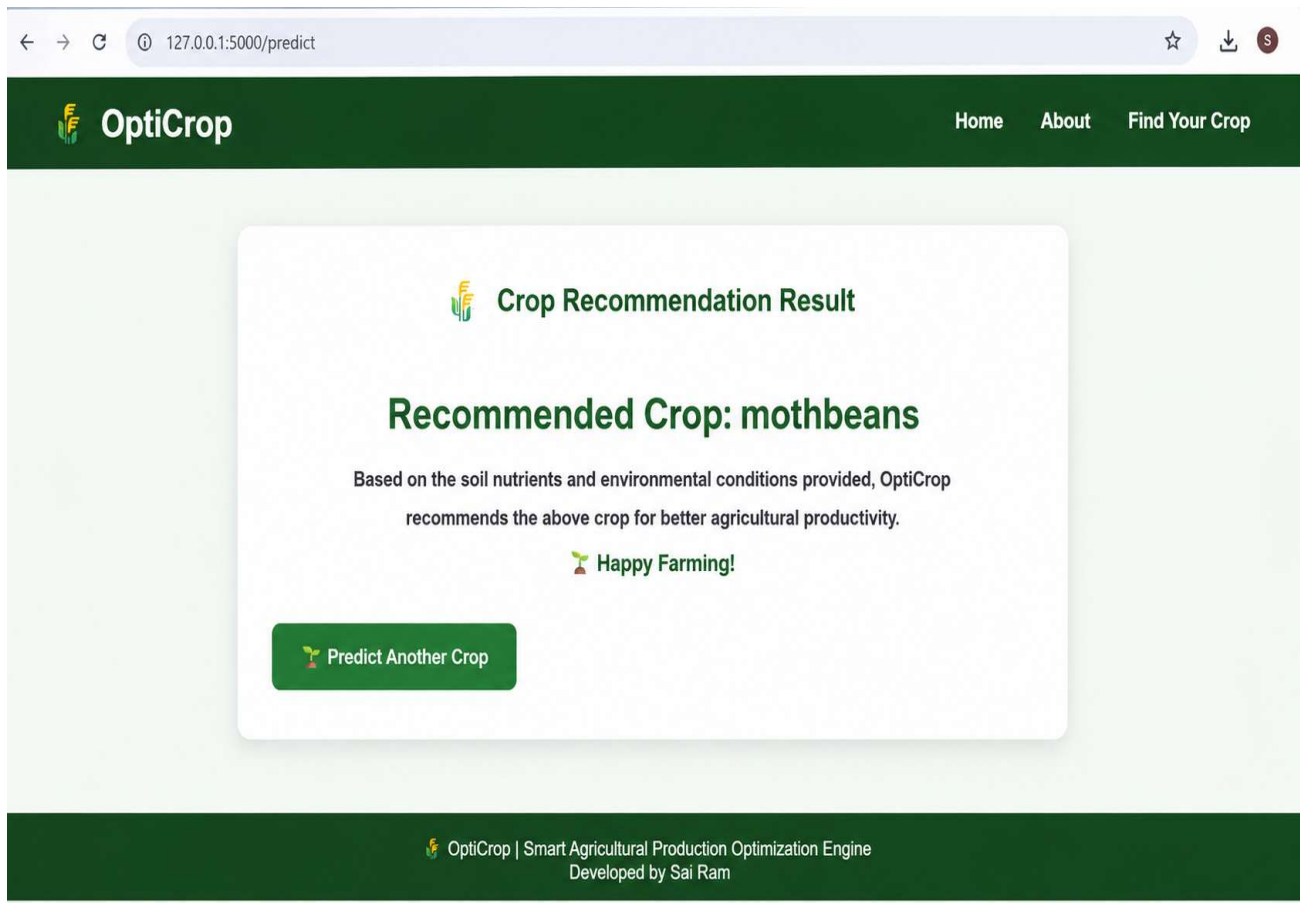
The screenshot displays a web browser window with the URL `127.0.0.1:5000/findyourcrop`. The page features a dark green header with the **OptiCrop** logo and navigation links for [Home](#), [About](#), and [Find Your Crop](#). The main content area is a light green background with a white form titled **Enter Agricultural Parameters**, which includes a small plant icon. The form contains seven input fields: Nitrogen (N), Phosphorous (P), Potassium (K), Temperature (°C), Humidity (%), pH Value, and Rainfall (mm). The pH and Rainfall fields include up and down arrow icons for selection. A prominent green button labeled **Predict Crop** is positioned at the bottom of the form.

Activity 4.3: Display Prediction Result

The prediction result page displays the crop recommended by the trained KNN model along with a message indicating successful prediction. Users can also navigate back to predict another crop.

Steps:

- **Step 1:** Create result.html.
- **Step 2:** Display the predicted crop.
- **Step 3:** Show a success message.
- **Step 4:** Add the Predict Another Crop button.



Milestone 5: Testing and Project Completion

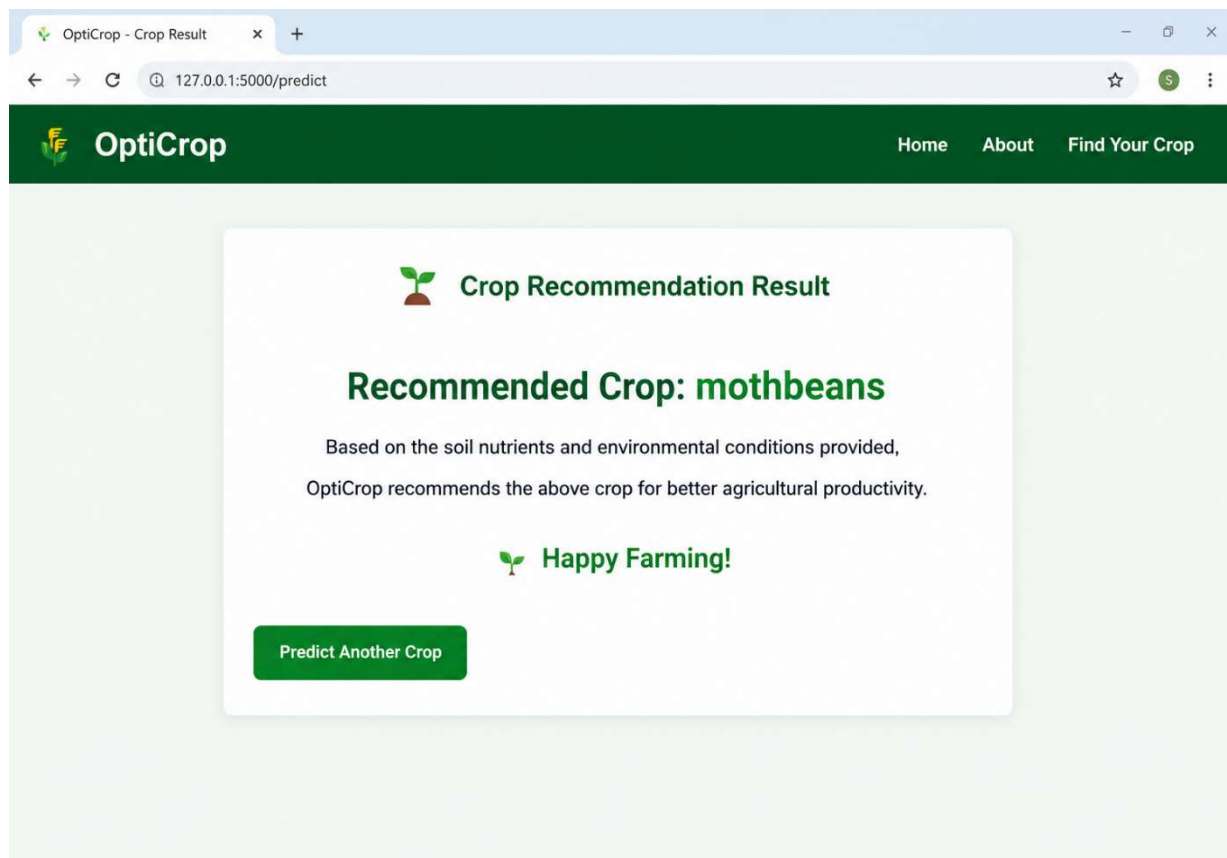
in this milestone, the Opti Crop application is tested to verify its functionality, performance, and prediction accuracy. Apache JMeter is used for basic performance testing, and the project is finalized with documentation and GitHub repository updates.

Activity 5.1: Functional Testing

The application is tested by entering different agricultural parameters to verify that the KNN model predicts the correct crop and all web pages function properly.

Steps:

- **Step 1:** Launch the Flask application.
- **Step 2:** Open the Home page.
- **Step 3:** Enter agricultural parameters.
- **Step 4:** Verify the predicted crop result.



Activity 5.2: Performance Testing using Apache JMeter

Apache JMeter is used to perform load testing on the Flask application. The Summary Report and Graph Results are analysed to ensure stable performance under multiple requests.

Steps:

- **Step 1:** Create a JMeter Test Plan.
- **Step 2:** Configure the HTTP Request.
- **Step 3:** Execute the test.
- **Step 4:** Analyse the Summary Report and Graph Results.

Activity 5.2: Performance Testing using Apache JMeter

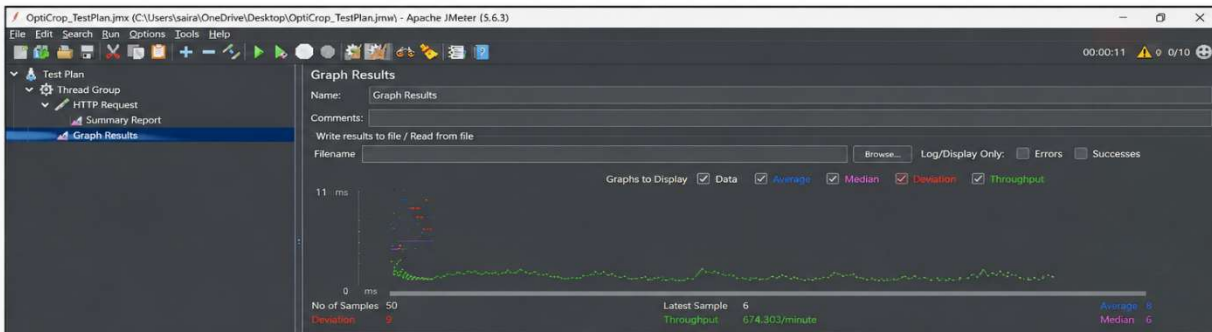


Figure 5.2 (a): JMeter Graph Results

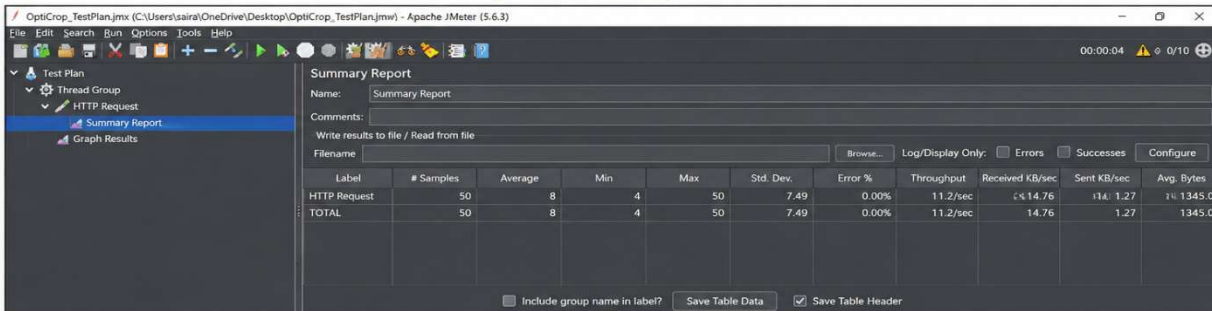


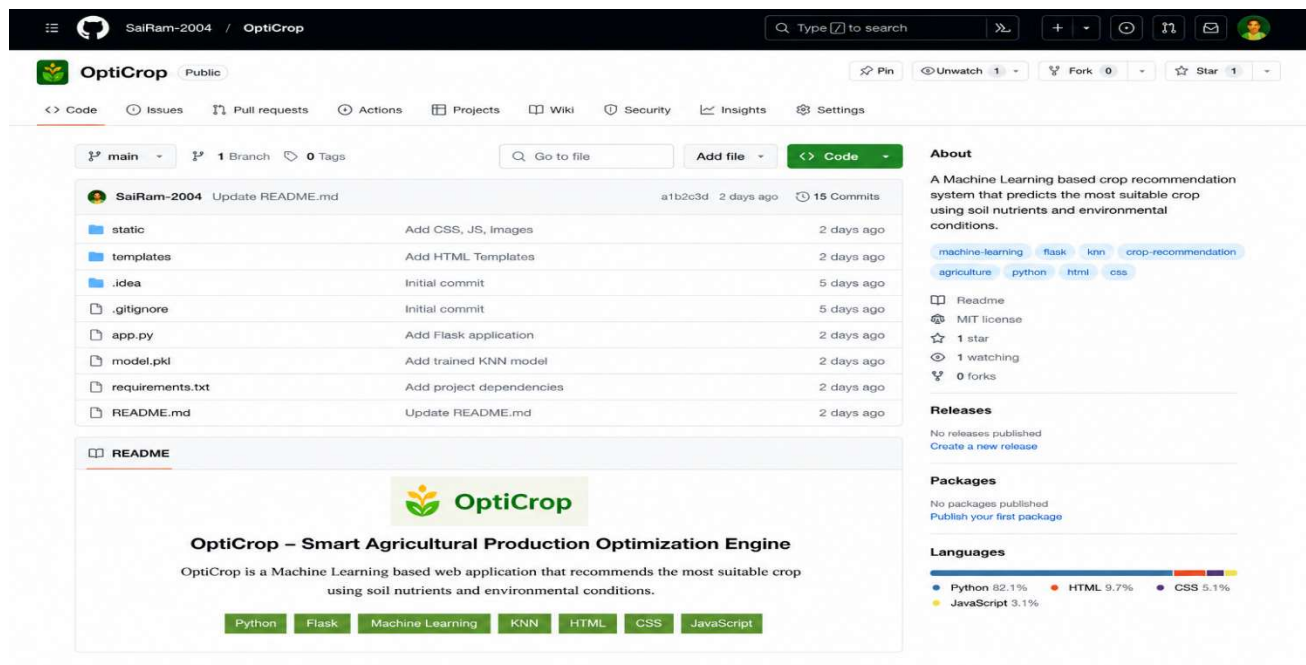
Figure 5.2 (b): JMeter Summary Report

Activity 5.3: Project Documentation and GitHub Repository

After successful testing, the project documentation is completed, and all source code, documentation, screenshots, and supporting files are uploaded to the GitHub repository for project submission.

Steps:

- **Step 1:** Prepare the project documentation.
- **Step 2:** Upload the project files to GitHub.
- **Step 3:** Verify the repository contents.
- **Step 4:** Copy the repository link for submission.



The screenshot shows the GitHub repository for 'OptiCrop' by user 'SaiRam-2004'. The repository is public and has 15 commits. The file list includes: static, templates, .idea, .gitignore, app.py, model.pkl, requirements.txt, and README.md. The README file is selected, showing the project description: 'OptiCrop – Smart Agricultural Production Optimization Engine'. It describes the application as a Machine Learning based web application that recommends the most suitable crop using soil nutrients and environmental conditions. The README also lists the technologies used: Python, Flask, Machine Learning, KNN, HTML, CSS, and JavaScript. The right sidebar shows the 'About' section with a description of the project, a list of tags (machine-learning, flask, knn, crop-recommendation, agriculture, python, html, css), and the 'Releases' and 'Packages' sections, both of which are currently empty.

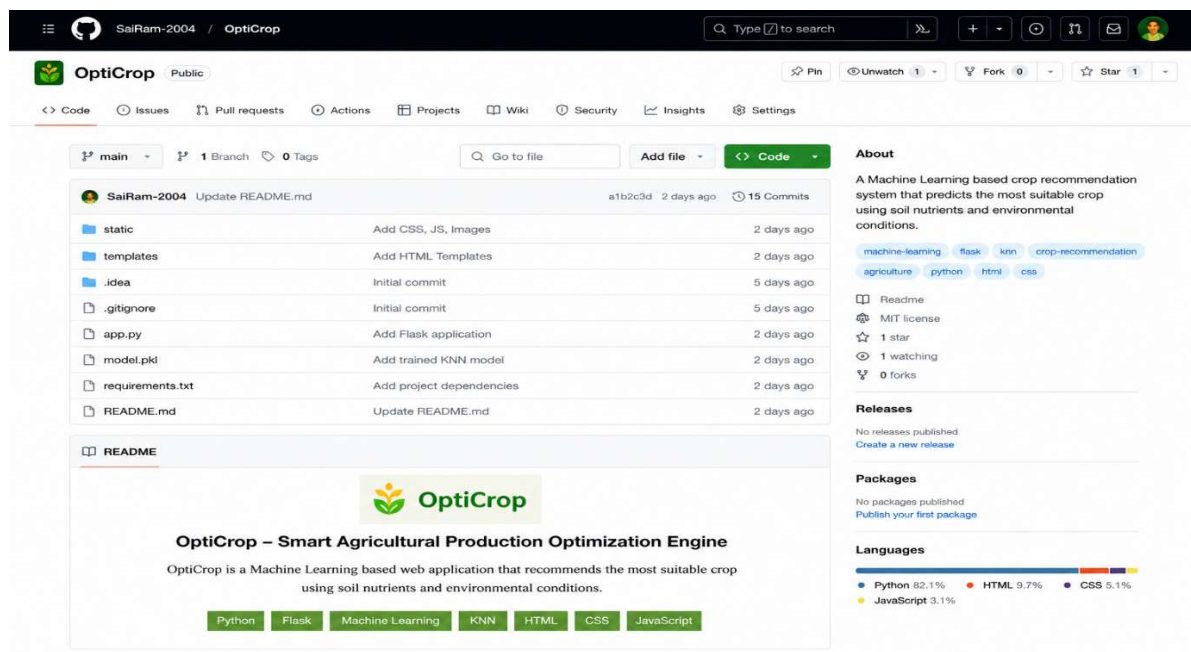
Milestone 6: Conclusion

The Opti Crop – Smart Agricultural Production Optimization Engine was successfully developed using Machine Learning and Flask. The KNN model accurately recommends suitable crops based on soil nutrients and environmental conditions. The web application provides a simple and user-friendly interface for farmers to obtain crop recommendations. Performance testing using Apache JMeter confirmed that the application responds efficiently. The project demonstrates the practical application of Machine Learning in smart agriculture and can be extended with advanced features in the future.

Activity 6.1: Project Completion

Steps

- **Step 1:** Verify all project functionalities.
- **Step 2:** Complete project documentation.
- **Step 3:** Organize all project files.
- **Step 4:** Finalize the GitHub repository



The screenshot shows a GitHub repository page for 'OptiCrop' by user 'SaiRam-2004'. The repository is public and has 15 commits. The file list includes: static, templates, .idea, .gitignore, app.py, model.pkl, requirements.txt, and README.md. The README file is selected, showing the project description: 'OptiCrop – Smart Agricultural Production Optimization Engine'. The description states: 'OptiCrop is a Machine Learning based web application that recommends the most suitable crop using soil nutrients and environmental conditions.' The README also features a logo for 'OptiCrop' and a list of technologies used: Python, Flask, Machine Learning, KNN, HTML, CSS, and JavaScript. On the right side, the 'About' section describes the project as a 'Machine Learning based crop recommendation system that predicts the most suitable crop using soil nutrients and environmental conditions.' It also shows the MIT license, 1 star, 1 watching, and 0 forks. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Languages' section shows a bar chart with the following data: Python 82.1%, HTML 9.7%, CSS 5.1%, and JavaScript 3.1%.

Activity 6.2: Future Enhancements

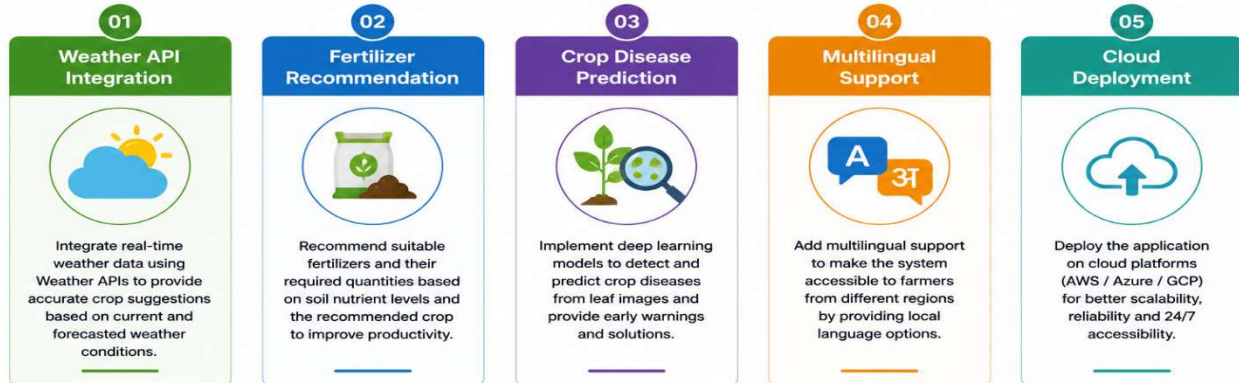
The project can be enhanced by integrating real-time weather APIs, fertilizer recommendation, disease detection, multilingual support, and cloud deployment to provide more intelligent agricultural assistance.

Steps

- **Step 1:** Add weather API integration.
- **Step 2:** Implement fertilizer recommendation.
- **Step 3:** Develop crop disease prediction.
- **Step 4:** Deploy the application to the cloud

Future Enhancements

The OptiCrop system can be enhanced with the following advanced features to provide more intelligent and comprehensive agricultural support.



Impact of Future Enhancements



More Accurate Predictions



Increased Crop Productivity



User Friendly & Accessible



Wider Reach Across Regions



Scalable & Reliable System



These enhancements will make OptiCrop a more powerful, intelligent and farmer-friendly agricultural solution.